

# Intelligent Agile Project Management Using Natural Language Processing

Dr. M R Suresh  
Professor  
Department of ISE  
PES College of Engineering

Bharath S  
UG Scholar  
Department of ISE  
PES College of Engineering

Chandrasekar L  
UG Scholar  
Department of ISE  
PES College of Engineering

Darshan M  
UG Scholar  
Department of ISE  
PES College of Engineering

Sanath Kumar S  
UG Scholar  
Department of ISE  
PES College of Engineering

**Abstract**—Software teams rarely talk about the hours that quietly disappear into bug triage reading through tickets, realising half of them describe the same issue, and manually consolidating what should never have been filed separately. Nor do they often discuss how sprint deadlines have a way of slipping past unnoticed until it is already too late to recover gracefully.

Both problems are common, both are costly, and neither has been well-addressed by the tools teams use every day. This paper describes a platform built to tackle both at once, inside a coherent Agile environment rather than as two disconnected add-ons.

The duplicate-detection side uses a fine-tuned Sentence-BERT model that reads bug reports for meaning not just matching words and quietly checks each new submission against everything already in the system. If a likely match surfaces, the reporter is warned before the ticket ever reaches a reviewer. The scheduling side uses a clean rule-based engine that watches deadlines in real time, bumps overdue tasks up the priority queue, and feeds trends into dashboards that are genuinely readable.

We trained the detection model on over 43,000 real-world bug reports, reaching 90.06% classification accuracy with F1-scores close to 0.90 for both classes. The whole system ships as a deployable full-stack application React on the front end, FastAPI in the middle, MongoDB at the back with no specialised DevOps setup required.

**Index Terms**—Natural Language Processing, Agile Project Management, Duplicate Bug Detection, Sprint Management, Sentence-BERT, FastAPI, Kanban

## I. INTRODUCTION

Ask any developer who has worked on a reasonably large project whether they have ever filed a bug that turned out to already exist, and the answer is almost always yes. Ask a triage engineer how much of their week goes into figuring out which incoming tickets are really the same issue, and the number is usually uncomfortable. Agile has done a great deal for software teams short cycles, regular feedback, customer proximity but it has not solved either of those problems, and most of the tooling built around it has not either.

The duplicate-bug issue is more expensive than it looks on the surface. Every time two people independently discover the same defect which happens constantly in active testing they each file their own report, phrased however makes sense to

them in the moment. Someone on the triage team then has to read both, recognise the overlap, and merge them. Scale that up across a large codebase and multiple testers, and a significant fraction of your engineering capacity ends up spent on work that produces no actual fixes. Studies put the share of duplicate reports in large projects at around 30% of all incoming submissions a lot of sorting for zero debugging.

Schedule slippage is a different kind of headache, but just as familiar. Sprint planning leans on estimates, and estimates are always a little wrong tasks run longer than expected, something urgent arrives from outside the sprint, a dependency gets blocked. What makes this particularly painful is that most project tools will not tell you a task is late until after it has already missed its deadline. By then the choices are limited: cut scope, delay the release, or push the team harder. Any of those options is worse than having caught the problem a week earlier and adjusted the plan while there was still room to adjust.

NLP has reached a maturity where it can genuinely help with the first problem. Models like Sentence-BERT do not just look for shared words between bug reports they encode what each report actually means and compare those meanings directly. Two reports about the same crash, written in completely different language, will end up sitting close together in the model's vector space. That is the behaviour you need for reliable deduplication. Combine that with modern frameworks like FastAPI and React, and there is no longer a good reason to choose between a research experiment and something a real team can deploy.

That is what we built: an Agile management platform where the AI does its work within the normal workflow. The submission of a bug triggers duplicate check. The dashboard surfaces which tasks are trending late. Alerts fire when something actually needs attention. The specific contributions this paper makes are:

- A working open-source Agile platform with duplicate detection and delay analytics built into the workflow from

the start, not added as an afterthought.

- A Sentence-BERT model fine-tuned on a large domain-specific bug corpus, including a full account of how the similarity threshold was calibrated and how the model was evaluated on a properly withheld test set.
- A rule-based delay detection mechanism that gives project managers clear and explainable signals without needing a training pipeline or historical project data to function.
- A detailed walkthrough of the architecture and implementation choices along with all the experimental results needed to reproduce or build on what we did.

The remainder of this paper is organized as follows: Section II reviews related work in duplicate bug detection, AI in Agile project management, and delay analytics. Section III describes the system architecture, module design, and implementation details. Section IV presents the experimental setup, dataset characteristics, model fine-tuning, and performance results. Section V discusses the implications, limitations, and potential future enhancements. Section VI concludes the paper.

## II. LITERATURE REVIEW

### A. Duplicate Bug Report Detection

People have been trying to automate duplicate bug detection for a while, and the approaches have changed considerably over time. Early methods relied on TF-IDF vectors and cosine similarity sometimes extended with structured fields like severity or component and they were reasonable for their era: fast, interpretable, no labelled training data needed. The fundamental problem was that they measured word overlap, not meaning. Two engineers filing reports about the same crash but using different terminology would look, to a TF-IDF system, like they were describing completely unrelated events.

Researchers responded by moving toward representations that could capture semantics. Latent Dirichlet Allocation offered topic-level abstractions. Word2Vec and GloVe gave distributed word representations that encoded some notion of meaning. Deep learning brought CNNs and RNNs capable of encoding an entire bug report into a dense vector. Then Transformer architectures arrived, and the self-attention mechanism - which considers every word in relation to every other word in the sequence pushed the state of the art further again.

The specific piece we build on is Sentence-BERT, developed by Reimers and Gurevych. It fine-tunes BERT using siamese and triplet networks, producing sentence embeddings that can be meaningfully compared with cosine similarity alone. The practical advantage is significant: you do not need to run both reports through the model together every time you check a pair which would be prohibitively slow when checking one new report against thousands of existing ones. What we add is a model trained on a large, purpose-built corpus, integrated into a live platform so it does real work on every submission rather than just scoring well on a benchmark.

### B. AI in Agile Project Management

Jira, Trello, and Azure DevOps have become infrastructure at this point so deeply embedded in how software teams work that it is hard to imagine operating without them. They are genuinely good at tracking issues, managing boards, and generating reports. Where they tend to fall short is in doing anything intelligent with the data they accumulate. The AI features that do exist usually come from third-party plugins and typically address narrow problems in isolation.

Academic research has set more ambitious targets - story-point estimation through machine learning, automated bug routing, sprint planning optimisation, risk flagging and the results in controlled settings have often been impressive. But the work rarely connects to itself. A model that predicts sprint velocity in a Jupyter notebook is not particularly useful to a team during actual sprint planning. The path from research result to something a working team encounters in their daily tool is almost always missing. What we are trying to offer here is that missing piece: a unified platform where several AI-driven capabilities work together within a realistic workflow.

### C. Delay Analytics in Software Projects

The literature on schedule overrun prediction covers a lot of ground regression, survival analysis, random forests, gradient boosting, neural networks with features ranging from planned-versus-actual effort to team velocity, dependency structure, and individual workload. Ensemble methods have held up well, and neural approaches have shown they can capture multi-factor interactions that simpler models miss.

Even so, the gap between a promising result in a research paper and a system that teams actually run day to day is large. Newly formed teams have no historical data there is nothing to train on. Project managers need to understand *why* something is flagged as risky, not just *that* it is; a model that cannot explain itself is hard to act on. Our response was to keep the delay module intentionally simple: if a task's deadline has passed and it is not marked Done, the system considers it delayed. One clean rule. It works on day one without any setup and it produces outputs that anyone can interpret.

### D. Research Gap and Contribution

Looking across duplicate detection, AI-augmented project management, and schedule analytics, a pattern shows up consistently: strong work on individual problems, very little integration between them, and a persistent gap between what researchers build and what teams can actually use. This platform was built to close that gap. The codebase is open source, the experiments are reproducible, and the system is meant to be run by real teams.

## III. METHODOLOGY

### A. System Architecture

The system is built in three layers with clean interfaces between them. The frontend is a React and Vite application styled with Tailwind CSS - componentised, responsive, and deliberately minimal so that engineers can focus on their work

rather than navigating the tool. The backend runs on Python 3.11 with FastAPI, which was the natural choice given its first-class async support, automatic API documentation, and solid performance under concurrent load without excessive boilerplate. MongoDB handles data persistence through the Motor async driver, and frontend-to-backend communication uses a REST API with JWT bearer tokens for authentication.

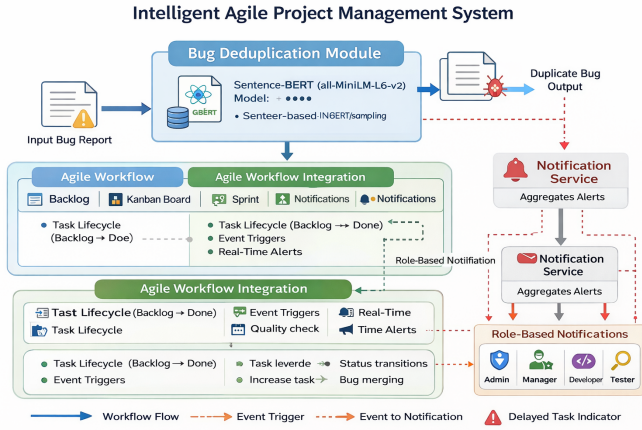


Fig. 1. High-level system architecture of the Intelligent Agile Project Management System.

Six backend modules handle distinct areas of responsibility:

- **Authentication and User Management** - JWT login with role-based access across four roles: Admin, Manager, Developer, and Tester.
- **Project and Sprint Management** - Full CRUD operations for projects, sprints, and backlog items.
- **Task Management** - A Kanban board with drag-and-drop status transitions, priority tracking, assignee management, and deadline monitoring.
- **Bug Management and Deduplication** - Bug submission, embedding generation and persistence, similarity search, duplicate flagging, and a manager-driven merge process.
- **Delay Analytics** - Automatic overdue detection, priority escalation, and chart data generation for dashboards.
- **Notifications** - Real-time alerts for task assignments, newly detected delays, bug merges, and sprint lifecycle events.

The AI model is loaded into memory once at startup. After that, every duplicate check is just an embedding lookup and a similarity calculation - no external API call, no warm-up time.

### B. Duplicate Bug Detection Module

The detection pipeline has two clearly separated phases: training happens offline; inference happens online.

1) *Model Fine-Tuning*: For training, we started from the all-MiniLM-L6-v2 checkpoint in the Sentence-Transformers library. It generates 384-dimensional sentence embeddings and sits at a useful balance point between accuracy and speed for an interactive application. We fine-tuned it using the `CosineSimilarityLoss` objective, which

pushes the model to bring embeddings of duplicate pairs closer together while keeping non-duplicate pairs further apart. Training ran for 2 epochs with a batch size of 16, a linear warmup over 115 steps, and an AdamW optimiser at a learning rate of  $2 \times 10^{-5}$ .

2) *Online Inference*: When a new bug is submitted, the inference process runs in five steps:

- 1) The report's title and description are joined into a single string.
- 2) The fine-tuned model encodes it into a 384-dimensional vector.
- 3) Stored embeddings for all existing bugs in the same project are pulled from MongoDB.
- 4) Cosine similarity is calculated between the new vector and each stored one.
- 5) If the top similarity score crosses 0.45, the reporter sees a warning with details of the potential match.

A manager can then review the flagged pair and merge the new report into the existing one, preserving the discussion thread and keeping the tracker clean.

Cosine similarity between two vectors  $u$  and  $v$  is defined as:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

### C. Agile Workflow Module

The task board has five columns - Backlog, To Do, In Progress, Review, and Done built on @hello-pangea/dnd, which works well with React 18. Dragging a card to a new column sends a PATCH request to the backend and updates the task status immediately. There is no save button to hit, and no uncertainty about whether the move went through.

Sprint management gets its own dedicated views for creating, editing, and closing sprints. Backlog items without a sprint can be dragged directly into an active sprint. Velocity and burndown figures are calculated server-side on each request and displayed on the dashboard, so the numbers are always current without requiring manual entry from anyone.

The permission model is tiered:

- **Admin**: Full system access, user management.
- **Manager**: Create projects, manage sprints, assign tasks, merge bugs.
- **Developer**: View and update assigned tasks and bugs.
- **Tester**: Report bugs, update bug status.

All key events generate notifications stored in MongoDB and retrieved by the frontend every 30 seconds, with unread counts shown in the navigation bar.

### D. Delay Handling Module

The delay logic is a single, transparent rule: if the current date is past a task's deadline and the task is not marked Done, set `is_delayed = true` and increase the priority by 2, up to a cap of 10. The endpoint `POST /tasks/recompute-delays` applies this across all active tasks. It fires automatically when the dashboard loads and can

also be called after any mutation that might change delay status.

Delayed tasks on the Kanban board receive a red left border and a small indicator icon visible without being disruptive. The dashboard shows three views: a pie chart of delayed versus on-time tasks, a bar chart of task distribution across the five status columns, and a line chart of task creation volume over the past six months. The first time a task gets flagged, both the assignee and the project manager receive a notification.

#### E. Technology Stack Summary

Table I summarizes the technologies used in each layer of the system.

TABLE I  
TECHNOLOGY STACK

| Layer           | Technology                                             |
|-----------------|--------------------------------------------------------|
| Frontend        | React, Vite, Tailwind CSS, Recharts, @hello-pangea/dnd |
| Backend         | Python 3.11, FastAPI, Pydantic v2                      |
| AI/NLP          | sentence-transformers (SBERT), scikit-learn, numpy     |
| Database        | MongoDB 7 (Motor async driver)                         |
| Auth            | JWT (python-jose), passlib (bcrypt)                    |
| Dev Environment | VS Code, Python virtualenv, npm                        |

## IV. EXPERIMENTAL SETUP AND RESULTS

#### A. Dataset Description and Preprocessing

The training data came from publicly available software repositories. Each record had four fields: `root_id` (grouping duplicate reports into a shared cluster), `disc_id` (a unique identifier per report), `dup_list` (listing all other report IDs in the same cluster), and free-text `title` and `description` fields.

The dataset contained 43,991 total records. Using the grouping fields, we identified 40,378 unique clusters, of which 37,963 were singletons (only one report) and 2,415 were duplicate clusters containing two or more reports. From these clusters, we generated positive (duplicate) pairs by pairing each report with every other report in the same cluster, yielding 6,603 positive pairs. An equal number of negative pairs (6,603) were sampled from different clusters, resulting in a balanced dataset of 13,206 pairs.

The dataset was split using stratified sampling to maintain class balance: 70% training (9,244 pairs), 15% validation (1,981 pairs), and 15% test (1,981 pairs).

#### B. Model Fine-Tuning and Threshold Selection

The `all-MiniLM-L6-v2` model was fine-tuned using the Sentence-Transformers training framework. Key hyperparameters are listed in Table II.

After training, we performed threshold selection on the validation set. Cosine similarity thresholds were scanned from -1 to 1 in increments of 0.01. For each threshold, we computed precision, recall, and F1-score. The threshold yielding the highest F1-score on the validation set was 0.45, with an F1-score of 0.9236.

TABLE II  
FINE-TUNING HYPERPARAMETERS

| Parameter           | Value                |
|---------------------|----------------------|
| Base Model          | all-MiniLM-L6-v2     |
| Embedding Dimension | 384                  |
| Loss Function       | CosineSimilarityLoss |
| Batch Size          | 16                   |
| Epochs              | 2                    |
| Warmup Steps        | 115                  |
| Optimizer           | AdamW                |
| Learning Rate       | $2 \times 10^{-5}$   |

#### C. Test Set Performance

The fine-tuned model was evaluated on the heldout test set using the selected threshold of 0.45. Table III reports the classification metrics for both classes.

TABLE III  
TEST SET PERFORMANCE AT THRESHOLD 0.45

| Class             | Precision | Recall | F1-Score |
|-------------------|-----------|--------|----------|
| Non-duplicate (0) | 0.9076    | 0.8920 | 0.8997   |
| Duplicate (1)     | 0.8937    | 0.9091 | 0.9014   |
| <b>Macro Avg</b>  | 0.9007    | 0.9006 | 0.9005   |
| <b>Accuracy</b>   | 0.9006    |        |          |

The confusion matrix for the test set is shown in Table IV. Out of 1,981 test pairs, the model correctly classified 1,784 pairs (884 true negatives and 900 true positives). There were 107 false positives (non-duplicates flagged as duplicates) and 90 false negatives (duplicates missed).

TABLE IV  
CONFUSION MATRIX AT THRESHOLD 0.45

|        |               | Predicted     |           |
|--------|---------------|---------------|-----------|
|        |               | Non-duplicate | Duplicate |
| Actual | Non-duplicate | 884           | 107       |
|        | Duplicate     | 90            | 900       |

#### D. Analysis and Discussion of Results

These numbers hold up well in context. Ninety percent accuracy with F1-scores near 0.90 on both classes is not the result of exploiting surface patterns the model is doing real semantic discrimination. Precision of 0.8937 on duplicates means roughly 89 out of every 100 warnings reflect a genuine match a low enough false alarm rate that triage teams should not start tuning them out. Recall of 0.9091 means the system catches more than 9 in 10 actual duplicates, which meaningfully cuts down on manual review load.

We chose 0.45 to balance precision and recall rather than sacrifice one for the other. Teams have the flexibility to shift it in either direction. If false positives are creating friction, a higher threshold reduces them. If catching every duplicate matters above all else say, because each missed one triggers a separate investigation track a lower threshold makes sense. The calibration process is documented well enough to repeat.

Looking at where the model fails is instructive. False positives tend to come from reports that share surface-level

symptoms two different bugs that both manifest as a login error, described in similar language but have different underlying causes. False negatives tend to involve duplicate pairs written in very different vocabularies, or cases where one report is too brief to provide meaningful signal. Incorporating structured metadata fields such as component, severity, and version is a reasonable next step for reducing both failure modes.

#### *E. System Performance and Usability*

A full duplicate check from text concatenation through embedding, MongoDB retrieval, and similarity calculation takes between 200 and 300 milliseconds on commodity hardware, even with several hundred existing bugs in the project. Reporters do not notice any lag before the warning appears. Dashboard data reloads on every page visit. Board interactions are immediate. Notifications land promptly when their trigger conditions are met.

### V. DISCUSSION

#### *A. Integration of AI into Agile Workflows*

Something genuinely encouraging about this project was how little the AI got in the way. A developer submitting a bug does not do anything differently the duplicate check happens in the background and only surfaces if there is actually something to show. Managers get a suggestion, not an instruction; they stay in control of what gets merged. The AI additions are strictly additive, which is probably a hard requirement for adoption in any team that already has a working rhythm.

The delay module reinforces this from a different direction. There is nothing to configure, no data to prepare. A manager looking at a red-bordered task card instantly knows what it means and why. That interpretability is not cosmetic people simply do not act on signals they do not understand, and they stop trusting systems that cannot be explained. The dashboard visualisations take that same clarity up to the sprint level, turning retrospective intuition into something that can be shown and discussed.

#### *B. Comparison with Existing Tools*

Jira is the obvious comparison point, and the main thing to note is that semantic duplicate detection is not part of it out of the box. Getting anything close requires either a third-party plugin or a custom implementation. Our platform has it from the first bug submission, and because the model lives in the codebase, teams can fine-tune it on their own data if the general model does not perform well enough for their specific domain. No licensing cost, no waiting on a vendor to ship a feature.

#### *C. Limitations and Threats to Validity*

We want to be straightforward about the limits of what this paper demonstrates:

- **Dataset Representativeness:** The training data came from publicly available repositories, which may not be representative of every domain. Teams in specialised

fields with unusual terminology, or those with unconventional reporting habits, may not see the same accuracy figures we report.

- **Threshold Sensitivity:** The 0.45 threshold was calibrated on validation data from the same distribution as the test set; moving to a new project should come with its own recalibration pass on a small local sample.
- **Rule-Based Delay Detection:** The delay module reports what has already happened rather than what is coming. That signal still has practical value late tasks need to be visible but it is not a prediction. A genuinely predictive component would allow earlier intervention, at the cost of needing enough historical sprint data to be trustworthy.
- **Evaluation Scope:** We have not run formal user studies, so the usability claims rest on our own design experience and judgment rather than controlled evaluation.

#### *D. Future Work*

Fine-tuning the SBERT model on organisation-specific or project-specific data is the most straightforward next step. The base model generalises reasonably well, but there is real room for improvement when the model has seen the kinds of reports a team actually writes. Adding an explainability layer something that highlights which phrases drove a duplicate flag - would give reporters more context and help them write better reports over time.

For scheduling, layering on a probabilistic prediction model would shift the system from reactive to proactive. Survival analysis and gradient boosting are both worth exploring; that step only makes sense once a team has enough sprint history to train on, which is exactly why we started with the rule-based approach rather than treating it as a compromise.

Connecting to GitHub, GitLab, and Slack would reduce tool-switching for teams already using those platforms. And a real user study with active Agile teams would produce concrete feedback on which features actually change behaviour versus which ones get ignored after the initial curiosity wears off the kind of grounding needed to prioritise what comes next.

### VI. CONCLUSION

We started with a simple premise: most software teams are losing real time to duplicate bug reports and late-discovered schedule slippage, and neither problem is well-served by the tools currently available. The platform we built addresses both within a single Agile environment React, FastAPI, MongoDB with a fine-tuned Sentence-BERT model handling duplicate detection and a transparent rule-based engine tracking delays.

Against a held-out test set drawn from a corpus of over 43,000 bug reports, the detection model achieved 90.06% accuracy with precision and recall both near 0.90. The threshold of 0.45 was chosen through systematic calibration on a separate validation set and applied without adjustment to the final evaluation. Delay detection is immediate and requires no setup tasks that miss their deadlines are flagged, prioritised, and surfaced on the dashboard automatically.

The broader takeaway is about what it actually takes to integrate AI into a team's workflow. It does not require an elaborate deployment, a model that nobody can explain, or a behaviour change that people will resist. It just requires getting the right information to the right person a little earlier than it would otherwise arrive. Done that way, the impact is real and the friction is low and that combination, more than any benchmark result, is what makes a tool worth using.

#### REFERENCES

- [1] T. Dingsøyr, N. B. Moe, and T. Dybå, "Agile Software Development: Trends and Future Directions," *IEEE Software*, vol. 37, no. 2, pp. 92–95, 2020.
- [2] S. Hoda, M. Salleh, and J. Grundy, "The Rise and Evolution of Agile Software Development," *IEEE Software*, vol. 37, no. 5, pp. 58–63, 2020.
- [3] P. Rodríguez et al., "Impact of Agile Practices on Software Quality and Productivity," *IEEE Software*, vol. 38, no. 2, pp. 63–71, 2021.
- [4] M. Paasivaara et al., "Scaling Agile in Large Organizations: Challenges and Practices," *IEEE Software*, vol. 38, no. 4, pp. 68–75, 2021.
- [5] R. Conboy and N. Carroll, "Agile Methodology: A Review and Research Agenda," *IEEE Computer*, vol. 54, no. 7, pp. 40–49, 2021.
- [6] A. Sharma et al., "Artificial Intelligence for Agile Project Management: A Systematic Review," *IEEE Access*, vol. 8, pp. 197807–197821, 2020.
- [7] P. Rodríguez et al., "Machine Learning for Predicting Sprint Performance in Agile Teams," *IEEE Software*, vol. 37, no. 4, pp. 44–51, 2020.
- [8] M. Choetkiertikul et al., "Predicting Issue Resolution Time Using Machine Learning," *IEEE Transactions on Software Engineering*, vol. 46, no. 6, pp. 590–606, 2020.
- [9] S. Banerjee et al., "AI-Based Analytics for Agile Software Development," *IEEE Access*, vol. 9, pp. 112450–112462, 2021.
- [10] R. G. Mäntylä et al., "Effort and Velocity Prediction in Agile Software Projects Using Machine Learning," *IEEE Software*, vol. 39, no. 2, pp. 52–59, 2022.